TP 5 : PROGRAMMATION ORIENTEE OBJET

Langage C++

Consigne : Ce TP est noté, les questions 1 à 14 sont à rendre sur Moodle et seront évaluées à la fin de séance, le reste du TP sera déposé sur Moodle au plus tard une semaine après la séance.

Dans ce TP, vous allez implémenter un système de manipulation de données tabulaires (DataFrame).

Partie 1 : Implémentation de la classe Colonne<T>

La classe Colonne<T> est une classe générique qui représente une colonne d'un DataFrame contenant des valeurs du type T, elle a une taille dynamique.

1. Déclarer une classe `Colonne<T>`, elle devra contenir au moins un tableau dynamique pour stocker ses valeurs de type `T` et un attribut `nom` pour donner un nom à la colonne
2. Ajouter un constructeur par défaut et un constructeur paramétré prenant un argument `std::string nom`
3. Ajouter un constructeur de copie permettant de copier une colonne
4. Ajouter une méthode `add(T valeur)` qui permet d'ajouter une valeur à la colonne
5. Ajouter une méthode `size_t taille()` const qui retourne la taille actuelle de la colonne
6. Surcharger l'opérateur `[]` afin de pouvoir accéder à une valeur à la position `i` de la colonne et permettant également de la modifier par assignation
7. Ajouter une méthode amie `afficher` qui affiche le contenu d'une colonne
8. Ajouter une méthode `vider()` qui vide le contenu de la colonne
9. Ajouter une destructeur à la classe
10. Tester la classe `Colonne<T>`

Partie 2 : Implémentation de la classe DataFrame

11. Déclarer une classe `DataFrame` qui stocke dynamiquement plusieurs colonnes (`Colonne<T>`)
    11.1. Ajouter 3 attributs permettant de stocker des colonnes int, double et string

```
std::vector<Colonne<int>> colonnesInt;
std::vector<Colonne<double>> colonnesDouble;
std::vector<Colonne<std::string>> colonnesString;
```

12. Ajouter 3 méthodes `ajouterColonne` permettant d'ajouter des colonnes int, double et string au dataframe en utilisant la surdéfinition. Il n'est pas possible d'avoir deux colonnes avec le même nom dans le dataframe, les fonctions `ajouterColonne` affichent une erreur si une colonne du même nom existe déjà
    12.1. Ajouter une méthode privée booléenne `colonneExiste(std::string nom)` permettant de faire cette vérification

13. Ajouter une méthode `supprimerColonne(std::string nom)` qui supprime une colonne du dataframe
14. Ajouter les methodes `nombreColonnes()` et `nombreLignes()` pour retourner respectivement le nombre de colonnes et de lignes, le nombre de ligne correspond à la taille de la colonne la plus longue
15. Ajouter un constructeur par défaut et un constructeur de copie
16. Ajouter un destructeur
17. Ajouter une methode template `Colonne<T>& getColonne(std::string nom)` pour récupérer une colonne par son nom
18. Ajouter une surcharge de l'opérateur [] (en utilisant une méthode template) permettant d'accéder également à une colonne par son nom:

```
template <typename T>
T& operator[](std::string nom)
```

19. Ajouter une surcharge de l'opérateur () permettant d'accéder et de modifier une valeur spécifique via (ligne, colonne), exemple: `df(1, "Age")`

```
template <typename T>
T& operator()(size_t ligne, std::string nom)
```

20. Ajouter une méthode amie afficher permettant d'afficher le contenu de chaque ligne du dataframe
21. Ajouter une méthode statique `merge` qui prend en argument deux dataframes et produit un dataframe résultat contenant les colonnes des deux dataframes source.